

La précision d'un ordinateur en programmation Python

PAKA Fabrice

10 juin 2025

1 Introduction

En informatique, Les ordinateurs utilisent un nombre fini de bits pour représenter les nombres, et cela influence la précision des calculs numériques.

2 Type numérique en python

Python propose plusieurs types numériques :

Type	Description
int	Entier de précision illimitée
float	Nombre à virgule flottante (64 bits)
Decimal	Précision arbitraire (via un module)
Fraction	Représentation exacte en fraction

3 Représentation des nombres entiers en utilisant les nombres binaires

Les nombres sont représentés sur un ordinateur par des bits, une succession de zeros et de uns.

Dans le cas des nombres entiers, la représentation est la suivante :

- Une bit pour le signe du nombre (0 si le signe est positif, 1 si il est négatif)
- une séquence de chiffres binaires

Exemple :

$$+5 = 0101, -5 = 1101$$

Il existe plusieurs formats de représentations des nombres entiers :

- format 32 bits : ces nombres sont représenter entre -2^{31} et $+2^{31}$
- format 64 bits : ces nombres sont représenter entre (-2^{63}) et (2^{63})

4 Représentation des nombres réels

Les nombres réels sont aussi représentés par des séquences binaires, qui sont séparées en trois composants :

- **Le signe(S)** : 1 bit indiquant le signe positif ou négatif du nombre
 - **l'exposant(E)** : Contrôle l'amplitude du nombre
 - **La mantisse (M)** : Contient les chiffres significatifs (précision)
- le nombre réel x est donc calculé comme suit : $x = S * M * 2^{E-e}$

5 Norme IEEE 754

La norme IEEE 754 définit comment les nombres sont représentés dans la plupart des ordinateurs modernes. Les deux formats principaux sont simple précision (32-bit) et la double précision (64-bit).

Cependant, cette représentation exacte sur un ordinateur n'est pas exacte, ce qui a parfois des conséquences sur les calculs scientifiques numériques. Par exemple :

- des erreurs d'approximations
- l'accumulation des erreurs dans des calculs itératifs
- la perte de chiffres significatifs utiles lors de la soustraction des nombres presque égaux

6 Exemples d'erreurs liées à la représentation machine des nombres réels

6.1 Erreurs d'approximation

Les nombres comme 0.1, 0.2, 1/3, n'ont pas de représentation exacte sur ordinateur et leurs approximations dans leurs représentations faussent légèrement leurs calculs :

```
1 x = 0.1
2 print(x)
3 s = x+x+x
4 print(s)
5 print(s == 0.3)
```

```
1 0.1
2 0.30000000000000004
3 False
```

Contrairement à 1. qui a une représentation exacte :

```
1 x = 1.
2 print(x)
3 s = x+x+x
4 print(s)
5 print(s == 3.)
```

```

1 1.0
2 3.0
3 True

```

6.2 l'accumulation des erreurs dans des calculs itératives

Considerons le calcul de la convergence de la série

$$S_n = \sum_{i=0}^n \frac{x^i}{i!}$$

```

1 from numpy import exp
2 def serie(x,n) :
3     term = 1.0
4     somme = term
5     for i in range(1,n):
6         term = term*x/i
7         somme = somme + term
8     return somme
9
10 x = -1.0
11 print("Pour x=",x, " serie=",serie(x,100), " exp(x)=",exp(x))
12 x = -100.0
13 print("Pour x=",x, " serie=",serie(x,1000), " exp(x)=",exp(x))

```

```

1 Pour x= -1.0  serie= 0.36787944117144245  exp(x)= 0.36787944117144233
2 Pour x= -100.0  serie= -2.9137556468915326e+25  exp(x)= 3.720075976020836e-44

```

On voit numériquement que cette série diverge pour $x < 0$ or qu'elle converge mathématiquement pour $x < 0$.

Cette erreur est souvent fixée en utilisant la propriété $e^{-x} = 1/e^x$

7 Code python pour calculer la précision d'une machine

```

1 def Precision():
2     """ calcul la precision machine """
3     eps = 1.0
4     while (1.+eps) > 1.0 :
5         eps = eps/2.
6     return 2*eps
7
8 print("Précision machine = ",Precision())

```